AI AUTOMATION · CASE STUDY

AI-Powered Passenger Document Automation System

An n8n workflow that turns passenger information received through a Telegram bot into a formatted PDF document, then delivers it back to the user in the same chat.

n8n | Telegram Bot | Docxtemplater | LibreOffice | AI-assisted processing

PROJECT AT A GLANCE

CATEGORY
AI Automation & Document Processing

PROJECT TYPE
Personal portfolio project (not a paying-client deployment)

WORKFLOW PLATFORM
n8n

MESSAGING INTERFACE
Telegram Bot, for input and PDF delivery

SUPPORTED INPUTS
Text, photos and documents

TEST STATUS
End-to-end test passed

CONTENTS

Case Study Outline

01 Project Overview
02 Problem Statement
03 Project Objectives
04 Solution Architecture
05 Workflow Logic
06 Technology Stack
07 Date and Time Handling
08 Testing and Validation
09 Technical Challenges and Learning
10 Current Limitations
11 Future Improvements
12 Conclusion

01 · OVERVIEW

Project Overview

The AI-Powered Passenger Document Automation System is an **n8n** workflow that receives passenger-related information through a **Telegram bot**, organizes it into structured fields, generates a formatted document, converts it to PDF, and sends the file back through Telegram.

I built it as my first AI automation project to practise designing and testing a multi-step workflow that combines messaging input, AI-assisted information processing, template-based document generation and file conversion. It is a **personal portfolio project**, not a production deployment for a paying client.

02 · PROBLEM

Problem Statement

Preparing passenger-related documents usually means collecting details from several input formats, moving them into a document template, converting the result to PDF and sharing it. Each step is repetitive, and handling the information consistently by hand is difficult.

This project explores whether those stages can be connected into one automated process, from receiving the information to delivering the finished PDF.

03 · OBJECTIVES

Project Objectives

- Accept passenger information as text, photos and documents through a Telegram bot.
- Organize the key details: client name, driver details, vehicle and registration, departure and destination, and passenger details.
- Apply Saudi date and time defaults when timing information is not provided.
- Populate a document template automatically and convert it to PDF.
- Deliver the final PDF to the user through Telegram.

04 · ARCHITECTURE

Solution Architecture

I built an n8n workflow that links every stage into one pipeline. A user sends information to the bot; the workflow identifies the input type, processes it, maps the results into template fields, generates the document, converts it to PDF and returns the file in the same chat.

Telegram Input → Input Routing → Information Processing → Field Mapping → Docxtemplater → LibreOffice PDF → Telegram Delivery

The tested messaging interface is **Telegram only**. WhatsApp integration has not been implemented.

05 · WORKFLOW

Workflow Logic

1. **Input handling.** A Telegram trigger receives incoming messages. A routing step checks the input type and sends text, photos and documents to their matching processing paths.
2. **File retrieval and processing.** Photo and document inputs are first retrieved so their content can be processed. Text inputs go through the configured AI-processing path.
3. **Information organization.** Extracted details are arranged into the exact fields the template expects: client name, driver details, vehicle and registration, departure and destination, and passenger details.
4. **Document generation.** **Docxtemplater** fills a Word template with the prepared fields and produces a completed DOCX file.
5. **PDF conversion.** **LibreOffice** runs headless to convert the DOCX file into a PDF.
6. **Final delivery.** The generated PDF is read back into the workflow as binary data and sent to the user through Telegram.

06 · STACK

Technology Stack

n8n
Workflow orchestration

Telegram Bot
Input handling and PDF delivery

AI processing node/model
Configured information-processing path

Docxtemplater
Template-based document generation

LibreOffice
DOCX-to-PDF conversion

File read/write nodes
Handling generated files

07 · TIMING

Date and Time Handling

The workflow includes a defaulting rule. When required timing information is missing from the input, the configured logic applies **Saudi date and time defaults** so document generation can continue instead of stopping on an incomplete submission.

This rule is part of the workflow design. It does not prove that every possible missing-field scenario has been tested.

08 · VALIDATION

Testing and Validation

An end-to-end test of the workflow completed successfully. The verified outcomes were:

- The workflow executed from start to finish.
- A PDF document was generated.
- The PDF was sent through Telegram.

These results show that the tested workflow can complete the document-generation and delivery process. **No time savings, accuracy percentage, production reliability or business impact is claimed**, because none have been measured.

09 · LEARNING

Technical Challenges and Learning

Connecting separate stages into one flow gave me practical experience in:

- Routing different input types to the right processing path.
- Structuring extracted data so downstream nodes receive predictable fields.
- Mapping fields into a document template.
- Handling DOCX and PDF files inside an automation workflow.
- Configuring headless file conversion.
- Passing binary file data between nodes.

The key lesson was to validate the **entire workflow** rather than checking individual nodes in isolation. A node can work alone and still fail when its output feeds the next stage.

10 · SCOPE

Current Limitations

- This is a portfolio prototype with a successful end-to-end test, not a production system.
- Telegram is the only implemented interface. WhatsApp integration has not been built.
- The successful test does not prove every input type, edge case or missing-information scenario has been validated independently.
- No time savings, accuracy rates or business impact have been established.

11 · ROADMAP

Future Improvements

Possible next steps, not features of the current implementation:

- Broader test coverage across input types and edge cases.
- Clearer validation for missing or unclear fields.
- More robust error handling.
- Additional messaging-platform integrations.
- A demo video, workflow screenshots and a sample PDF using fictional or anonymized data.

12 · SUMMARY

Conclusion

This project demonstrates a working automation flow that takes passenger information through Telegram, processes it with AI assistance, generates a document from a template, converts it to PDF and delivers it back to the user. It reflects my ability to design, connect, debug and test a multi-step **n8n** workflow involving document automation and file handling.

> It is my first AI automation project and a foundation for more robust, better-tested workflows. Its scope is clear: a **personal prototype** with a successful end-to-end test, built to develop and show practical automation skills.